Qualified-majority clauses and targeted forgetting for clause-constrained Tsetlin machines on associative processors

Anonymous authors

Abstract

In-memory Tsetlin machine (TM) accelerators store one clause per row, so the clause budget sets the array area and energy. The contribution in artificial intelligence is two learning modifications for small clause budgets. Qualified-majority (QM) clauses fire when at most a fixed fraction of their included literals is violated. Targeted forgetting (R1) restricts the Type Ib feedback of a non-firing clause to literals that are false on the sample. The application in engineering is in-memory TM hardware. On an associative-processor TM, QM uses the threshold of an approximate-search content-addressable memory, and R1 adds one compared bit. Projected inference latency is unchanged, and training needs 36 to 44 % more cycles than the original design. We evaluate both on 19 public datasets with protocols fixed in advance and Holm-corrected Welch tests. Against the vanilla TM as first tuned, R1 wins 24 of 48 cells, all on sparse text, and a density rule (R1 below 3 % density, QM above) wins 5 of 12 cells on six unseen datasets without a loss. ~~A control then strengthened only the vanilla TM by lowering its specificity and literal budget. Against it, R1 still wins 10 sparse cells by 3.0 to 11.7 percentage points (pp) and loses 1. The best QM-family arm beats the better of this baseline and R1 in 6 dense cells by 1.2 to 5.5 pp and loses in 2. Out of sample, the density rule is not confirmed (2 wins, 3 losses). The baseline tuning grid matters more than either modification.~~ Controls then lowered the specificity and literal budget of the vanilla TM and the specificity of R1 and QM. With all arms re-tuned, R1 wins 19 sparse cells by 1.5 to 17.2 percentage points (pp) and loses 8 dense cells. QM wins 8 cells by 1.2 to 10.9 pp and loses 2. Out of sample, the density rule is not confirmed (3 wins, 2 losses). Neither R1 nor QM works without the literal budget. Unequal tuning grids changed the conclusions more than either modification did.

Keywords: Tsetlin machine, associative processor, content-addressable memory, approximate search, in-memory computing, interpretable machine learning

1. Introduction

The Tsetlin machine (TM) [1] classifies with conjunctive clauses of Boolean literals, learned by teams of finite-state automata. Its inference uses only AND, NOT and counting. This makes it a natural fit for in-memory hardware, where a clause maps to one memory line and is evaluated in place [2, 3]. AP(TM) [3] implements both training and inference of a TM on an associative processor (AP) [4]. It evaluates all clauses with one compare and updates all selected clauses with parallel writes. Its latency therefore depends only weakly on the number of clauses. Its area and energy are not: every clause occupies a row, and every compare and write activates every row.

The clause budget is thus the main hardware cost of an in-memory TM. A standard TM, however, needs many clauses. A clause either fires or does not, and a single violated literal silences it. With few clauses per class, a TM loses accuracy because each clause must be specific enough to avoid false positives and general enough to fire on unseen samples.

This paper asks whether two simple modifications can recover accuracy at small clause budgets without changing the hardware substantially, and how robust any gain is to the tuning of the vanilla baseline.

- **Qualified-majority (QM) clauses.** A clause fires when at most a fraction $1 - \theta$ of its included literals is violated. The vanilla TM is the case $\theta = 1$.
- **Targeted forgetting (R1).** When a clause of the target polarity does not fire, standard Type Ib feedback moves every automaton of the clause toward exclusion with probability $1/s$. R1 applies this forgetting only to the literals that are false on the sample, and leaves the satisfied literals untouched.

The two rules suit an associative processor well. QM clause evaluation is a Hamming-distance-tolerant search, which approximate-search CAMs support with a programmable threshold [5, 6]. R1 conditions an existing feedback write on the input bit, which the AP already broadcasts.

Contributions.

1. We define relative QM clauses, a polarity-aware variant, and R1, and we combine them with the literal budget of [7] (Section 3).
2. We map QM, QP and R1 onto AP(TM) with an approximate-search CAM, and we project the cost with the analytical model of [3] (Section 4).

3. We evaluate all combinations against the vanilla TM with protocols fixed in advance, on 13 public datasets and 20 encodings from 0.34 % to 54 % input density, with five test seeds per arm and family-wise error control. We then test a density rule, R1 below 3 % input density and QM above, on six further datasets (Sections 5 to 6.2).
4. We repeat every contrast against a stronger vanilla TM, tuned over a smaller specificity value and a smaller literal budget (Section 6.3), and we re-tune it in a budget sweep up to 160 clauses per class (Section 6.5). ~~The modified arms are not re-tuned. Most of the original R1 gains vanish.~~ Re-tuning only the vanilla TM removes most of the original R1 gains. Offering the same specificity to R1 and QM restores most of them (Section 6.3). R1 keeps significant gains on sparse many-class text and QM on dense image and signal data, but the density rule is not confirmed out of sample. An ablation shows that neither modification works without the literal budget (Section 6.4). We report these negative results in full.

2. Background

2.1. Tsetlin machine

A TM for K classes and n Boolean features uses $M = 2n$ literals, the features and their negations. Each class has C clauses, half of positive and half of negative polarity. Each clause j owns one Tsetlin automaton (TA) per literal. A TA with 2^N states includes its literal in the clause in the upper half of its states and excludes it in the lower half. Clause j outputs

$$o_j = \bigwedge_{k \in I_j} \ell_k, \tag{1}$$

where I_j is the set of included literals and ℓ_k the value of literal k on the sample. The class score is the number of firing positive clauses minus the number of firing negative clauses. The prediction is the class with the highest score.

Training presents each sample to its target class and to one randomly drawn other class [1, 8]. Each clause of these classes receives feedback with a probability that falls as the class score approaches a margin T. Type I feedback goes to clauses whose polarity agrees with the sample's membership in the class. If such a clause fires, Type Ia moves the TAs of true literals toward inclusion and those of false literals toward exclusion. If it does not fire, Type Ib moves every TA of the clause toward exclusion with probability $1/s$. Type II feedback goes to firing clauses of

the other polarity and moves the TAs of excluded false literals toward inclusion, so that the clause stops firing. The literal budget L of [7] limits clause growth: a firing clause with more than L included literals receives Type Ib instead of Type Ia. Type II feedback is not limited, so $|I_j|$ can exceed L.

2.2. AP(TM)

AP(TM) [3] stores the clauses of all classes in the rows of a CAM-based associative processor. Following the AP(TM) row layout, a row holds an N-bit TA ~~state~~ ($N = 8$) and a~~n includ~~etemporary bit for each of the M literals, and $51 + \lceil \log_2 K \rceil$ control bits. In the *compare-tm* operation, the input vector drives the search lines and only cells whose TA is in the include half can discharge the match line. The match line stays high only if every included literal is true, which implements Eq. (1) in all rows at once. Training runs in three phases per class update: stochastic row selection, stochastic mask generation, and feedback execution. The last phase visits the literals one at a time and updates the TA counters of all selected rows in parallel. Per class update it takes [3]

$$\text{Cycles} = 127 + 236 \log_2 s + (8N + 16)M. \tag{2}$$

Inference takes $5K + 1$ cycles plus at most $\lceil \log_2 m \rceil$ cycles of reduction over m rows, and the energy of each operation is proportional to the number of rows [3].

2.3. Approximate-search CAM

A NOR-type CAM discharges the match line through every mismatching cell [9]. The discharge rate therefore grows with the number of mismatches. Hamming-distance-tolerant CAMs sense this rate against a programmable reference and declare a match when the number of mismatches does not exceed a threshold [5, 10]. A 65 nm 128-kbit implementation with a tunable Hamming-distance threshold was demonstrated in silicon [6]. We assume the simplest case, a threshold common to all rows of a search.

3. Qualified-majority clauses and targeted forgetting

3.1. Relative qualified-majority clauses

Let v_j be the number of included literals of clause j that are false on the sample, and $n_{\text{inc},j} = |I_j|$. A relative QM clause fires if

$$v_j \le \tau_j = \left\lfloor (1 - \theta)\, n_{\text{inc},j} \right\rfloor, \tag{3}$$

with $\theta \in (0, 1]$. For $\theta = 1$, Eq. (3) reduces to Eq. (1). The tolerance grows with the clause, so a long clause may miss a few literals while a short clause stays strict. The QM output replaces o_j everywhere, in inference and in all feedback decisions during training. ~~Without a literal budget, QM clauses grow large and lose accuracy, so every QM configuration in this paper uses one.~~ Without a literal budget, QM clauses grow to thousands of literals and lose accuracy (Section 6.4), so every QM configuration in this paper uses one.

The polarity-aware variant (QP) keeps negated literals strict and applies the tolerance to the included positive literals only. It targets sparse inputs, where a positive literal is rarely true and is therefore the usual cause of a violation.

3.2. Targeted Type Ib feedback (R1)

In standard Type Ib feedback, a non-firing clause of the target polarity forgets every literal with probability $1/s$. This includes literals that the sample satisfies. R1 restricts forgetting to the violated literals:

$$\Pr\left[\mathrm{TA}_{j,k} \downarrow\right] = \begin{cases} 1/s & \text{if } o_j = 0 \text{ and } \ell_k = 0, \\ 0 & \text{if } o_j = 0 \text{ and } \ell_k = 1. \end{cases} \tag{4}$$

A clause that fires but exceeds the literal budget keeps standard Type Ib. All other feedback is unchanged. Like QM, R1 needs the literal budget (Section 6.4). The rationale is specific to sparse inputs. There, most negated literals are true on every sample. Standard Type Ib erodes them whenever a clause fails to fire, although they did not cause the failure. R1 keeps them, so a clause can retain the negated literals that exclude other classes.

3.3. Arms

Table 1 lists the six arms. Each arm is a family of two or three configurations, and the best member is chosen on validation data.

4. Hardware embodiment on AP(TM)

Figure 1 shows one AP(TM) row with the additions. The literal cells, the TA counters and the three training phases are those of AP(TM) [3]. We add a per-row counter of included literals, a small number of bias cells, and a programmable match-line reference. The polarity-aware variant QP adds a second counter and two flag bits. All costs in this section are projections from the AP(TM) model, not measurements.

Table 1: Arms. L: literal budget. θ: QM threshold. Each arm is a family. Its best configuration is selected on validation data.

Arm	Clause	Type Ib	Configurations
TM	AND	standard	no budget; $L = 64$
TMR	AND	R1	$L = 32$; $L = 64$
QM	relative QM	standard	$(\theta, L) = (0.90, 32)$, (0.95, 32), (0.95, 64)
QMR	relative QM	R1	$(\theta, L) = (0.90, 32)$, (0.95, 32)
QP	polarity-aware	standard	$\theta = 0.01, 0.10$; $L = 64$
QPR	polarity-aware	R1	$\theta = 0.01, 0.10$; $L = 64$

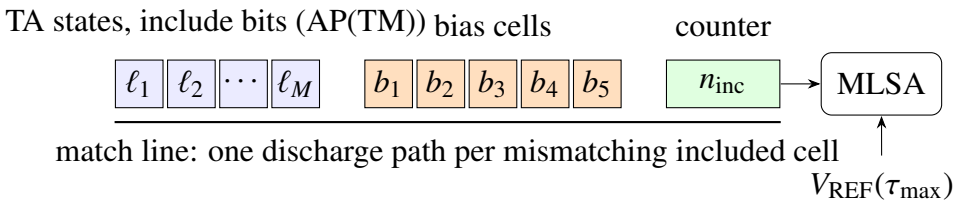


Figure 1: AP(TM) row with the QM additions (shaded orange and green). Bias cell b_t, $t = 1, \ldots, \tau_{\max}$, mismatches on every search if $\tau_j < t$, so the global threshold $\tau_{\max}$ yields the per-row tolerance τ_j. Drawing not to scale.

4.1. QM clause evaluation

Compare-tm discharges the match line through every included literal that is false, so the number of discharge paths is v_j. A threshold compare-tm with reference τ therefore declares a match if $v_j \le \tau$. With a threshold common to all rows, Eq. (3) still needs a per-row tolerance τ_j. We add $\tau_{\max}$ bias cells per row. Bias cell b_t, $t = 1, \ldots, \tau_{\max}$, is written to mismatch on every search if $\tau_j < t$, and to match otherwise. Row j then has $v_j + (\tau_{\max} - \tau_j)$ mismatches, and the global threshold $\tau_{\max}$ gives

$$v_j + \tau_{\max} - \tau_j \le \tau_{\max} \iff v_j \le \tau_j. \tag{5}$$

The bias cells must be refreshed when $n_{\text{inc},j}$ changes, which happens only during training.

Clamped tolerance. Type II feedback is not limited by the literal budget, so $n_{\text{inc},j}$ and τ_j are not bounded. The hardware therefore clamps the tolerance at $\tau_{\max} = 5$:

$$\tau_j^{\text{hw}} = \min\left(\left\lfloor (1-\theta)\, n_{\text{inc},j} \right\rfloor, \tau_{\max}\right). \tag{6}$$

This equals Eq. (3) for clauses with up to 59 ($\theta = 0.90$) or 119 ($\theta = 0.95$) included literals. We inspected the largest clause every five epochs in the six cells where QM wins, for tuning seed 1 and test seed 101. Its size ranged from 35 to 76 literals, within these limits, at all checkpoints but one: on UCI HAR with 5 clauses per class and $\theta = 0.90$, it held 60 literals at epoch 5 of seed 101, where the clamp gives a tolerance of 5 instead of 6. The QM cells of the density-rule test (Section 6.2) and the other test seeds were not inspected. The accuracy results of Section 6 were obtained with the unclamped rule of Eq. (3). We did not re-run them with the clamp.

4.2. QP clause evaluation

Let $n_{\text{pos},j}$ be the number of included positive literals of clause j, and v_j^+ the number of them that are false on the sample. A QP clause fires if none of its included negated literals is false and $v_j^+ \le \tau_j^+ = \lfloor (1-\theta)\, n_{\text{pos},j} \rfloor$. The two conditions have different tolerances, so one threshold compare-tm cannot evaluate them together. QP therefore uses two compare-tm operations with modified search keys. A literal cell discharges the match line only if its literal is included and its search-line value is false. Driving the search lines of a group of literals to true thus removes that group from the compare. First, an exact compare-tm with the positive-literal columns driven true tags the rows whose included negated literals are all true. One write stores this tag in a row flag F_1. Second, a threshold compare-tm

with the negated-literal columns driven true counts only v_j^+. With the bias cells set from τ_j^+ as in Section 4, it tags the rows with $v_j^+ \leq \tau_j^+$. One write stores this tag in a row flag F_2. Third, an exact compare on $F_1 = F_2 = 1$ gives the clause outputs. QP evaluation thus takes three compares and two writes instead of one compare, four additional cycles per evaluation. This is below 0.02 % of the cycles of Eq. (2) for every encoding.

An arm uses either QM or QP, so QP reuses the $\tau_{\max} = 5$ bias cells of QM. They are refreshed from $n_{\text{pos},j}$ instead of $n_{\text{inc},j}$. For $\theta = 0.01$ and 0.10, the clamped tolerance equals τ_j^+ for clauses with up to six included positive literals. A saturating counter of $c' = 3$ bits per row holds $n_{\text{pos},j}$. It is incremented in the recount pass of Section 4.3, for the $M/2$ positive literals only, with the include tag and the increment enable of $n_{\text{inc},j}$. This adds $(4c' + 3)\, M/2$ cycles per class update, and the bias refresh takes at most $\tau_{\max}(2c' + 2) = 40$ cycles. Over all encodings, QP with $L = 64$ adds 49.4 to 53.1 % to the training cycles of Eq. (2). This is 9.0 to 9.4 percentage points more than the literal budget alone. The counter and the two flags widen the row by at most 1.33 %.

4.3. Included-literal counter and literal budget

The literal budget needs $n_{\text{inc},j} \leq L$ before each Type I decision, and the bias cells need τ_j^{hw}. Each row holds a saturating counter of c bits that resolves $n_{\text{inc},j}$ up to $\max(L + 1, \lceil \tau_{\max}/(1 - \theta) \rceil)$, so $c = 6$ or 7. After the feedback phase of every class update, the controller recounts $n_{\text{inc},j}$. It clears the counters, and for each literal it tags the rows whose TA is in the include half and increments their counters with the saturating increment of AP(TM) (Algorithm 1 of [3], $2c + 1$ compares and $2c + 2$ writes). This takes $(4c + 5)M + 1$ cycles. For $L = 2^k$, the budget test is two compares and one write per class update. The extra Type Ib branch for firing clauses over the budget adds one compare and one write per literal. The bias refresh takes at most $\tau_{\max}(2c + 2)$ cycles per class update.

4.4. R1

AP(TM) sets the decrement enable of Type Ib with a compare on the row's polarity, output and stochastic mask bit [3]. R1 adds the literal's input bit, which is already on the search lines, to this compare. It adds no cycle and one compared bit.

4.5. Projected cost

Table 2 summarizes the projected overhead. Inference uses one threshold compare-tm in place of the exact one. Its cycle count is unchanged under the assumption below. We assume that a threshold compare takes one cycle and the

energy that AP(TM) [3] assumes for an exact compare. The circuit-level cost of the programmable reference is not modeled. Training is dominated by the recount, which grows with M like the feedback phase itself. It adds 35.8 to 43.7 % of the cycles of Eq. (2), depending on c and s. These overheads are relative to AP(TM) as published, which has no literal budget. The overhead is caused by the literal budget, which R1 and QM need(Section 6.4), and not by the QM threshold itself. The vanilla TM selected on validation in our experiments also uses a literal budget in most cells (Section 6), so its AP(TM) mapping would need the same counter. Against that baseline, R1 adds no training cycles and QM adds only the bias refresh, at most 80 cycles per class update. An incremental update that detects include transitions during the feedback phase would avoid most of the recount when few transitions occur. We did not evaluate it.

Table 2: Projected AP(TM) overhead of the additions, over all 26 encodings and $s \in \{2.5, 10\}$. Training cycles relative to Eq. (2). Row width relative to the AP(TM) row.

Addition	Inference	Training cycles	Row width
R1	0 cycles	0 cycles	0 bits
Literal budget (c-bit counter)	0 cycles	+35.8 to +43.7 %	+c bits
QM threshold and bias cells	0 cycles	$\leq$ 80 cycles	+5 bits
All but QP, worst case	0 cycles	+43.7 %	$\leq$ +0.940 %
QP with $L = 64$ (all its additions)	+4 cycles	+49.4 to +53.1 %	$\leq$ +1.33 %

The area and energy of AP(TM) scale with the number of rows, $C \cdot K$ [3]. A configuration that reaches the accuracy of the vanilla TM with fewer clauses per class therefore saves array area and search energy in proportion. Section 6.5 measures this ratio.

5. Experimental setup

5.1. Datasets and encodings

Table 3 lists the datasets. MNIST [11], Fashion-MNIST [12], UCI Adult [13] and IMDb [14] use the Boolean encodings of AP(TM) [3]. UCI HAR [15] uses ten thresholds per feature at the training quantiles 5 to 95 %, and UCI Phishing [16] one-hot codes its categorical features. The text sets are R8 and R52 [17], 20 Newsgroups [18], Sector [19], RCV1 [20] and LEDGAR [21, 22]. Dorothea [23] is a sparse binary drug-discovery set with 100,000 features. R8, R52 and LEDGAR are encoded from raw text as a binary bag of words: lowercase alphabetic tokens

of two or more characters, NLTK English stop words [24] removed, and document frequency of at least two. Sector, 20 Newsgroups and RCV1 come from the LIBSVM collection [25], and their features are binarized as presence indicators. For all text sets and Dorothea, chi-square selection of V features uses the training part only. Several sets appear at several vocabulary sizes V, so that density varies within a dataset. Validation data are the official split if one exists, otherwise a stratified 10 % of the training split (15 % for R8, R52 and Dorothea). The official Dorothea validation set serves as its test set, and the UCI Phishing test set is a stratified 20 % split. The RCV1 test set is a random sample of 50,000 documents of the official test file. The six datasets marked † were added after study 1 to test the density rule (Section 6.2): AG News [26], TREC-6 [27], Emotion [28], USPS [29], Letter [30] and Satimage [31]. The three added text sets use the raw-text encoding above. TREC-6 keeps stop words, because its classes are defined by question words. Letter, USPS and Satimage come from the LIBSVM collection [25] and use ten thresholds per feature at the training quantiles 5 to 95 %, as UCI HAR.

5.2. *Protocol*

The evaluation has three parts. Study 1 (Section 6.1) compares all arms on 20 encodings. Study 2 tests a density rule on six new datasets (Section 6.2) and sweeps the clause budget (Section 6.5). The baseline controls (Section 6.3) strengthen the vanilla TM. They were designed after all Study 1 and Study 2 results were known, in response to a review, and their protocol was written before their runs. A symmetric control and a literal-budget ablation (Sections 6.3 and 6.4) were designed after the baseline controls, in response to a second review. Their protocol was also written before their runs. Each protocol was written before its runs and amended only before the affected runs. The protocols are internal documents, not external registrations, and are released with the code and all logs. The two amendments of Study 1 added datasets, reduced the grid for them, and excluded three encodings for run time and memory. The Study 1 protocol specifies a 10 % validation split. The R8, R52 and Dorothea encodings, prepared earlier, use 15 %. The Study 2 protocol also specified a comparison with the reference code of the Fuzzy-Pattern TM [32]. It was run but is not reported, because that work is not peer reviewed and its graded vote does not map to one threshold search (Section 7). Clause budgets are $C = 5, 10, 20$ per class with $T = 3, 5, 10$ for the eight original encodings, and $C = 5, 10$ for the twelve added ones. Specificity s is tuned per arm over $\{2.5, 5, 10\}$, or $\{2.5, 10\}$ for the added encodings. All runs train for 40 epochs. For each dataset, budget and arm, the configuration and s with the highest validation accuracy at the last epoch on tuning seed 1 are selected (ties: smaller s,

Table 3: Datasets and encodings, ordered by training-set density. Active: mean number of true input bits per sample. C: clause budgets per class tested. †: added for the out-of-sample density rule.

Dataset	K	Train/val/test	Bits n	Density (%)	Active	C
20NG (V=20k)	20	14,341/1,594/3,993	20,000	0.34	68	5/10
R52 (full)	52	5,576/984/2,569	10,705	0.38	40	5/10/20
Sector (V=1k)	105	5,770/642/3,207	1,000	0.55	6	5/10
Sector (V=5k)	105	5,770/642/3,207	5,000	0.59	30	5/10
Sector (V=20k)	105	5,770/642/3,207	20,000	0.61	122	5/10
Dorothea (V=5k)	2	680/120/350	5,000	0.61	31	5/10
R8 (V=5k)	8	4,745/838/2,207	5,000	0.67	33	5/10/20
R52 (V=5k)	52	5,576/984/2,569	5,000	0.68	34	5/10/20
LEDGAR (V=5k)	100	60,000/10,000/10,000	5,000	0.69	34	5/10
20NG (V=5k)	20	14,341/1,594/3,993	5,000	0.80	40	5/10
Dorothea (full)	2	680/120/350	100,000	0.91	909	5/10
RCV1 (V=5k)	51	14,007/1,557/50,000	5,000	0.98	49	5/10
20NG (V=1k)	20	14,341/1,594/3,993	1,000	1.28	13	5/10
R52 (V=1k)	52	5,576/984/2,569	1,000	1.35	13	5/10/20
IMDb	2	22,500/2,500/25,000	5,000	2.10	105	5/10/20
UCI Adult	2	35,165/3,908/9,769	162	13.49	22	5/10/20
MNIST	10	54,000/6,000/10,000	784	15.14	119	5/10/20
UCI Phishing	2	7,959/885/2,211	68	44.12	30	5/10
UCI HAR	6	6,616/736/2,947	5,610	49.65	2786	5/10
Fashion-MNIST	10	54,000/6,000/10,000	784	54.07	424	5/10/20
Emotion†	6	16,000/2,000/2,000	5,000	0.11	6	5/10
TREC-6†	6	4,906/546/500	3,089	0.23	7	5/10
AG News†	4	108,000/12,000/7,600	5,000	0.28	14	5/10
Letter†	26	10,500/4,500/5,000	160	42.59	68	5/10
USPS†	10	6,561/730/2,007	2,560	47.94	1227	5/10
Satimage†	6	3,104/1,331/2,000	360	48.59	175	5/10

then the first listed configuration). The selected settings are then trained with seeds 101 to 105 and evaluated once on the test split. The best QM-family arm and the best non-QM arm of each cell are also chosen on validation.

Baseline controls. In Study 1 the vanilla TM selected $s = 2.5$, the smallest grid value, in 42 of 48 cells, and its family lacked the budget $L = 32$ that TMR and QM use. The controls therefore define an extended vanilla family, $\mathrm{TM}_{\mathrm{ext}}$: no budget, $L = 64$ and $L = 32$, each with the original s grid plus $s = 1.5$. $\mathrm{TM}_{\mathrm{ext}}$ is selected with the rule above on all 48 Study 1 and 12 Study 2 cells, reusing the original vanilla runs, and confirmed with seeds 101 to 105 where its selection changes. Every contrast is then recomputed against $\mathrm{TM}_{\mathrm{ext}}$. Only the vanilla TM is strengthened: the modified arms keep their original grid, selections and test runs. The symmetric control then adds $s = 1.5$ to the grid of TMR, QM and QMR in each cell. These arms are re-selected with the same rule, reusing their original runs, and confirmed with seeds 101 to 105 where the selection changes. QP and QPR are not re-tuned. The symmetric control covers the 48 Study 1 and the 12 Study 2 cells. The literal-budget ablation removes L from the TMR and QM settings selected in the symmetric control, keeps s and T, and runs seeds 101 to 103. An arm counts as working without the budget only if it has no significant loss against the budgeted arm, its median difference exceeds -1 pp, and it is within 1 pp in at least three quarters of the cells. This criterion was fixed before the runs. The controls can only raise the validation accuracy of the baseline. On test data it fell in 7 of 60 cells, by at most 1.3 pp. In the budget sweep, the vanilla TM is re-tuned at each budget from 40 to 160 clauses per class over $\{L = 32, L = 64\} \times \{1.5, 2.5\}$ and its original setting, and confirmed with seeds 101 to 103. In the symmetric control, TMR and QM are re-tuned in the same way at each budget from 40 to 160 clauses per class, over their original sweep setting and every member of their family at $s = 1.5$, with seed 1 on validation. A changed selection is confirmed with seeds 101 to 103. [Harel: Why is IMDb excluded from the symmetric control and the re-tuned sweep?]

5.3. Statistics

The unit of analysis is the seed. Accuracy differences are given in percentage points (pp). Each contrast compares the five test accuracies of two arms with Welch's t-test [33]. Holm's procedure [34] controls the family-wise error at 0.05 within each contrast over the 48 cells. The protocol fixed six contrasts: TMR−TM, QM−TM, QP−TM, QMR−TMR, QPR−TMR, and best QM-family arm minus best non-QM arm. The split of cells at 3 % density was chosen after Study 1. Any threshold between 2.1 and 13.5 % separates the Study 1 encodings identically. The

relation of gain to input density and to the number of active features per sample is descriptive (Spearman correlation over cells, which are not independent within a dataset).

5.4. Implementation

All arms use pyTsetlinMachine 0.6.6 [8], patched for QM, QP and R1. With $\theta = 1$, standard feedback and the original feedback sampler, the patched code reproduces the original bit for bit, in predictions and in all TA states. For speed, all runs, including the vanilla TM, select the literals that receive feedback with an independent Bernoulli$(1/s)$ mask per literal instead of the original sampled-count scheme. The marginal selection probability is the same, but the random stream differs. Inputs are packed into 32-bit words once per run. The patches, the experiment scripts and the analysis code were written with Claude (Anthropic), a generative AI assistant, under the direction of the authors. Study 1 comprised 1,560 tuning runs and 1,440 test runs, and the baseline controls 426 tuning runs and 345 test runs, and the symmetric control, the ablation and the sweep with re-tuned TMR and QM 564 tuning runs and 961 test runs, all on two CPU cores.

6. Results

6.1. R1 and QM against the vanilla TM

This section and the next report the contrasts against the vanilla TM as tuned in the Study 1 protocol. Section 6.3 repeats them against a stronger vanilla TM. Table 4 summarizes the six contrasts, and Fig. 2 plots the gains against input density. The per-cell results are in Table A.9 (Appendix A).

Table 4: Study 1 contrasts over 48 cells against the original vanilla TM (test accuracy, five seeds, Welch, Holm). Sparse: training density below 3 % (35 cells). Dense: the other 13.

Contrast	Wins	Losses	Median gain (pp)		Spearman
			sparse	dense	ρ (density)
TMR − TM	24	6	+6.26	−4.37	−0.63
QM − TM	5	0	−0.83	+1.23	+0.24
QP − TM	0	2	+0.45	−0.52	−0.43
QMR − TMR	7	3	−3.45	+5.33	+0.62
QPR − TMR	0	2	−0.10	−1.02	−0.31
Best QM − best other	6	0	−0.04	+1.05	+0.48

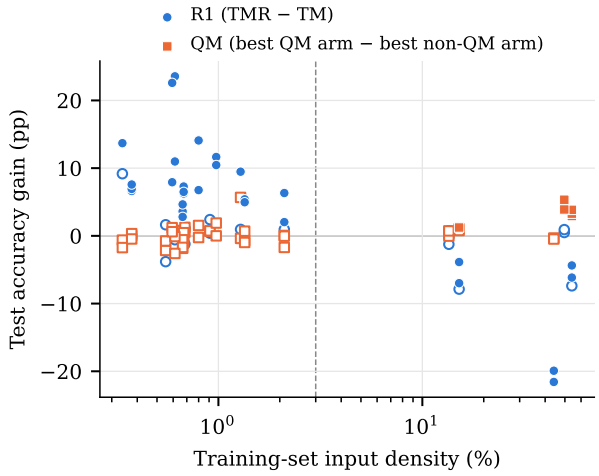


Figure 2: Test accuracy gain per (dataset, budget) cell against training-set input density. Circles: R1 (TMR − TM). Squares: QM (best QM-family arm − best non-QM arm). Filled markers: significant after Holm correction. Open markers: not significant. Dashed line: the 3 % density boundary of the rule in Section 6.2.

R1. R1 is the larger effect. It is significantly better than the vanilla TM in 24 cells and worse in 6. All significant gains are on sparse inputs, where the median gain is +6.26 pp and 32 of 35 cells are positive. The largest gains occur on Sector, 20 Newsgroups and RCV1, which combine many classes with very sparse inputs: +22.6 and +23.6 pp on Sector with 10 clauses per class. All significant losses are on dense or one-hot inputs: MNIST and Fashion-MNIST at 10 and 20 clauses, and UCI Phishing, where R1 loses about 20 pp. The gain falls with density (Spearman $\rho = -0.63$, $p = 1.3 \times 10^{-6}$). It is not correlated with the number of active features per sample ($\rho = -0.06$, $p = 0.71$).

QM. QM is a smaller, dense-input effect. The best QM-family arm is significantly better than the best non-QM arm in 6 cells and worse in none. All six are dense: MNIST at 10 clauses (+1.23 pp), Fashion-MNIST at 5, 10 and 20 (+2.93, +3.28, +3.84 pp) and HAR at 5 and 10 (+5.33 and +3.90 pp, with R1). On sparse inputs the best QM arm is not significantly different from the best non-QM arm: median −0.04 pp, 17 of 35 cells positive, no significant difference. The QM−TM gain is weakly correlated with density ($\rho = +0.24$, $p = 0.10$) and more strongly with the number of active features per sample ($\rho = 0.37$, $p = 0.01$). QM with R1 recovers much of the loss that R1 causes on dense inputs (QMR−TMR, median +5.33 pp on dense cells) but loses to TMR on sparse ones (median −3.45 pp).

Negative results. The polarity-aware variant does not help. QP and QPR have no significant win against their baselines and two significant losses each. At the validation stage, QPR appeared 1.1 to 5.4 pp better than TMR in 10 of the 15 R52 and 20 Newsgroups cells, and worse in 4. This did not replicate on fresh seeds (QPR−TMR median on sparse cells −0.10 pp), which illustrates the optimism of selecting the best member of an arm on one validation seed.

6.2. Out-of-sample density rule

Study 1 suggests a simple rule: use R1 if the training density is below 3 % and QM otherwise. We fixed this rule and the six new datasets of Table 3 before running them. The hypothesis fixed in advance is that the arm chosen by the rule beats the vanilla TM. The rule counts as confirmed if it has more significant wins than losses and no significant loss. Table 5 gives the results for $C = 5$ and 10.

Table 5: Out-of-sample density rule on six datasets not used in study 1 (test accuracy, mean of five seeds). Rule: R1 (TMR) below 3 % density, QM otherwise. Δ rule: rule arm minus TM. Δ other: the other family minus TM, for reference. Bold: significant after Holm correction over the 12 cells of the respective contrast.

Dataset	Dens. (%)	C	TM (%)	Rule (%)	Δ rule (pp)	Δ other (pp)
Emotion	0.11	5	44.96	R1 52.80	+7.84	**−12.93**
Emotion	0.11	10	49.46	R1 44.63	−4.83	**−14.26**
TREC-6	0.23	5	49.40	R1 56.28	+6.88	−5.80
TREC-6	0.23	10	61.00	R1 56.12	−4.88	**−16.04**
AG News	0.28	5	47.28	R1 74.92	**+27.64**	−7.89
AG News	0.28	10	59.18	R1 82.23	**+23.05**	−14.66
Letter	42.59	5	62.30	QM 63.66	+1.36	+1.03
Letter	42.59	10	70.28	QM 71.02	+0.74	−1.62
USPS	47.94	5	82.36	QM 87.02	**+4.65**	**+2.61**
USPS	47.94	10	85.92	QM 89.38	**+3.46**	+0.75
Satimage	48.59	5	81.16	QM 81.96	+0.80	+0.06
Satimage	48.59	10	82.90	QM 84.63	**+1.73**	−0.30

Against the original vanilla TM, the rule is confirmed. The rule arm is significantly better than the vanilla TM in 5 of 12 cells and never significantly worse. It is ahead in 10 of 12 cells (one-sided sign test, $p = 0.019$). R1 gains 27.6 and 23.1 pp on AG News with 5 and 10 clauses. QM gains 4.7 and 3.5 pp on USPS and 1.7 pp on Satimage with 10 clauses. On Emotion and TREC-6, the rule arm is not significantly different from TM. On these two small sets, both arms vary widely

across seeds, with standard deviations of 7.0 to 12.0 pp for R1 and 0.8 to 8.8 pp for the vanilla TM.

The rule also matters. Applying the other family instead gives 1 significant win and 3 significant losses. QM on the three sparse sets loses 5.8 to 16.0 pp, three times significantly. In study 1, QM had no significant loss in any cell, but the sparsest study 1 encoding has 0.34 % density. The three new text sets, at 0.11 to 0.28 %, show that QM can hurt very sparse inputs, which supports routing them to R1. R1 on the dense sets is closer to neutral here than in study 1, and it is significantly better than TM on USPS with 5 clauses. Against the stronger baseline of Section 6.3, the rule is not confirmed.

6.3. Baseline controls

Table 6 repeats the contrasts of Sections 6.1 and 6.2 against the extended vanilla family $\mathrm{TM}_{\mathrm{ext}}$ (Section 5). The selection of the vanilla TM changed in 45 of 48 Study 1 cells and in all 12 Study 2 cells. $\mathrm{TM}_{\mathrm{ext}}$ chose $s = 1.5$ in 38 and 12 of these cells, and $L = 32$ in 21 and 5. Its test accuracy exceeds that of the original vanilla TM by a median of 1.7 pp in Study 1 (2.0 pp on sparse and 0.6 pp on dense cells) and 3.6 pp in Study 2. The largest increases are 15.0 pp on Sector with 20,000 words and 10 clauses per class, and 28.3 pp on AG News with 5. Because $s = 1.5$ is the smallest value tried, the vanilla TM may not be fully tuned even now.

R1. Against $\mathrm{TM}_{\mathrm{ext}}$, R1 wins 10 Study 1 cells and loses 7. All wins are on sparse many-class text: RCV1 (+11.7 and +10.0 pp), Sector (+7.6 and +8.5 pp), R52 (+3.0 to +5.5 pp) and R8 (+3.3 pp). On sparse cells R1 still wins 10 times and loses once, on 20 Newsgroups with 1,000 words and 5 clauses per class, but its median gain falls from +6.26 to +1.32 pp. It still loses on dense cells, by up to 23.4 pp on UCI Phishing. In Study 2, R1 has no significant win against $\mathrm{TM}_{\mathrm{ext}}$ and loses 4 cells, all with 10 clauses per class: AG News (4.3 pp), TREC-6 (15.4 pp), Letter (5.3 pp) and USPS (1.7 pp). Tuning the vanilla TM recovers most of the original R1 gain on these sets: on AG News the vanilla TM itself gains 28.3 and 27.4 pp from the extended grid. With $s = 1.5$ offered to R1 as well, R1 wins 19 Study 1 cells against $\mathrm{TM}_{\mathrm{ext}}$ and loses 8. All wins are on sparse text, by 1.5 to 17.2 pp: Sector, RCV1, 20 Newsgroups, R52, R8, LEDGAR and IMDb (1.5 pp with 20 clauses per class). All losses are on dense cells: MNIST and Fashion-MNIST at all three budgets (4.2 to 7.7 pp) and UCI Phishing (18.1 and 19.4 pp). R1 selected $s = 1.5$ in 35 of 48 cells. Offering it raised the accuracy of R1 significantly in 7 cells and lowered it in none (median +1.16 pp). In Study 2, R1 with $s = 1.5$ wins on Emotion (+19.8 and +25.0 pp) and loses on Letter, AG News and USPS with 10 clauses (5.3, 2.2 and 1.7 pp).

Table 6: Contrasts against the original vanilla TM and against the extended vanilla family TM_{ext} (test accuracy, five seeds, Welch, Holm within each contrast and baseline). W/L: significant wins and losses. Med.: median gain in pp. Sparse: training density below 3 %. ~~Only the vanilla TM is re-tuned.~~ Original: neither arm re-tuned. Extended: only the vanilla TM re-tuned. Both re-tuned: $s = 1.5$ also offered to TMR, QM and QMR. QP and QPR are not re-tuned. "Best other" is the better of TMR and the vanilla TM on validation. The Study 2 TMR and QM rows are descriptive additions to the control protocol.

		All cells		Sparse		Dense	
Contrast	Baseline	W/L	Med.	W/L	Med.	W/L	Med.
Study 1, 48 cells							
TMR − TM	original	24/6	+2.60	24/0	+6.26	0/6	−4.37
	extended	10/7	+0.15	10/1	+1.32	0/6	−5.43
	both re-tuned	19/8	+1.61	19/0	+3.67	0/8	−5.43
QM − TM	original	5/0	−0.11	0/0	−0.83	5/0	+1.23
	extended	4/13	−2.67	0/12	−4.23	4/1	+0.62
	both re-tuned	8/2	−0.57	4/2	−1.32	4/0	+0.95
QP − TM	original	0/2	+0.13	0/0	+0.45	0/2	−0.52
	extended	0/15	−1.25	0/11	−1.15	0/4	−1.89
Best QM − best other	original	6/0	+0.06	0/0	−0.04	6/0	+1.05
	extended	6/2	−0.02	0/0	−0.04	6/2	+1.23
	both re-tuned	4/1	−0.64	0/1	−1.58	4/0	+0.95
Study 2, six new datasets, 12 cells							
Rule arm − TM	original	5/0	+2.59	2/0	+7.36	3/0	+1.55
	extended	2/3	−0.04	0/2	−2.47	2/1	+0.77
	both re-tuned	3/2	+0.85	2/1	+8.52	1/1	+0.17
TMR − TM	original	3/0	+0.89	2/0	+7.36	1/0	+0.40
	extended	0/4	−1.26	0/2	−2.47	0/2	−1.26
	both re-tuned	2/3	−0.67	2/1	+8.52	0/2	−1.26
QM − TM	original	3/3	−2.53	0/3	−13.59	3/0	+1.55
	extended	2/6	−6.15	0/5	−22.73	2/1	+0.77
	both re-tuned	1/6	−6.08	0/5	−18.64	1/1	+0.17

QM. Plain QM loses to TM_{ext} in 13 Study 1 cells and wins 4. All losses but one are on sparse text, where QM had no significant loss against the original baseline. On dense cells QM wins 4 times (Fashion-MNIST at 5, 10 and 20 clauses and MNIST at 10, by 1.2 to 3.5 pp) and loses once (UCI Phishing at 10 clauses, 2.3 pp). The best QM-family arm against the better of TM_{ext} and TMR wins 6 dense cells and loses 2: it adds QMR on UCI HAR (+5.5 and +2.4 pp) and loses on UCI Phishing with 10 clauses (2.3 pp) and UCI Adult with 20 (0.4 pp). In Study 2, QM beats TM_{ext} on USPS (+1.7 and +1.0 pp). On the dense sets it loses only on Letter with 10 clauses (2.9 pp), and it loses 5 cells on the three text sets. QP loses 15 cells and wins none. With $s = 1.5$ offered to QM as well, QM wins 8 Study 1 cells and loses 2. It wins on Fashion-MNIST at all three budgets and on MNIST at 10 clauses (1.2 to 3.5 pp), and on Sector with 5,000 and 20,000 words (6.8 to 10.9 pp). It loses on Sector with 1,000 words (6.5 pp) and on R52 with 1,000 words and 20 clauses (3.6 pp). Offering $s = 1.5$ raised the accuracy of QM significantly in 8 cells, all but one on Sector and 20 Newsgroups, and lowered it in none. The best QM-family arm wins 4 cells, on MNIST and Fashion-MNIST, and loses 1, on 20 Newsgroups with 1,000 words (5.7 pp). In Study 2, QM with $s = 1.5$ wins only on USPS with 10 clauses (1.0 pp). It loses 6 cells: five on the three text sets (16.2 to 23.1 pp) and Letter with 10 clauses (2.8 pp).

Density rule. As a descriptive check outside the control protocol, the rule applied to Study 1, where it was derived, beats TM_{ext} in 14 cells and loses in 2, with significance taken from the Holm-corrected TMR and QM contrasts. On the six unseen datasets of Study 2, it wins 2 cells and loses 3, and it is ahead in 6 of 12 (one-sided sign test, $p = 0.61$). By the criterion fixed in Section 6.2, the rule is not confirmed against the stronger baseline. With both arms re-tuned, the rule applied to Study 1 wins 23 of 48 cells and loses none. On the six unseen datasets it wins 3 cells (Emotion with 5 and 10 clauses, +19.8 and +25.0 pp, and USPS with 10, +1.0 pp) and loses 2 (Letter and AG News with 10 clauses, 2.8 and 2.2 pp). It is ahead in 7 of 12 cells (one-sided sign test, $p = 0.39$). By the same criterion, the rule is again not confirmed.

Caveat. The controls favor the baseline by design. ~~The modified arms also selected the smallest *s* in most cells (TMR in 42, QM in 30 of 48), so they might also gain from *s* = 1.5. We did not run this symmetric control. The results above therefore bound what the modifications add over a vanilla TM tuned on a finer grid, under tuning budgets that are not equal.~~ The symmetric control gives all arms the same s grid. TM_{ext} can also choose no literal budget, which TMR and QM cannot. Section 6.4 shows that this option lowers their accuracy. TMR, QM and QMR selected $s = 1.5$, the smallest value offered, in 35, 27 and 28 of 48 Study 1

cells.

6.4. Literal-budget ablation

The ablation tests whether R1 and QM need the literal budget L. It removes L from the TMR and QM settings of the symmetric control and keeps s and T (Section 5.2). Table 7 compares the test accuracy with and without L on the same seeds.

Table 7: Literal-budget ablation (test accuracy, seeds 101 to 103, Welch, Holm within each row). No budget − budget: the same arm and setting without and with L. W/L: significant wins and losses. Med.: median difference in pp. Within 1 pp: cells with an absolute difference of at most 1 pp. Literals per clause: median over cells of the mean number of included literals per clause after training. Study 1: 48 cells. Study 2: 12 cells.

		No budget − budget				Literals per clause	
Study	Arm	Cells	W/L	Med.	Within 1 pp	Budget	No budget
1	TMR	48	0/39	−37.83	2	53.5	3786.9
1	QM	48	0/28	−31.75	3	28.4	2568.6
2	TMR	12	0/10	−45.11	0	65.7	1900.3
2	QM	12	0/4	−7.41	1	23.1	1259.9

Neither arm works without the budget. Without L, R1 loses significantly in 39 of 48 Study 1 cells and 10 of 12 Study 2 cells, with median losses of 37.8 and 45.1 pp. QM loses in 28 and 4 cells, with median losses of 31.8 and 7.4 pp. Neither arm wins a cell. The median number of included literals per clause rises from 23 to 66 with the budget to 1,260 to 3,790 without it. On UCI Phishing, where the clauses stay below the budget, removing L changes the test accuracy by at most 0.14 pp. Against $\mathrm{TM}_{\mathrm{ext}}$, R1 without L loses 43 Study 1 and 12 Study 2 cells, and QM without L loses 28 and 9. Both rules weaken the pressure that removes literals from a clause. R1 no longer forgets the satisfied literals of a non-firing clause. QM lets a clause fire with violated literals, so the clause receives Type Ia instead of Type Ib feedback. The literal budget therefore limits clause growth for both rules, and its counter (Section 4.3) is a necessary part of their hardware cost.

6.5. Clauses saved at equal accuracy

The budget sweep extends the eight encodings tested at $C = 5$, 10 and 20 to $C = 40$, 80 and 160. TMR and QM keep the configuration and s selected at $C = 20$ in Study 1, with $T = C/2$, and run with seeds 101 to 103. The sweep also has re-tuned TMR and QM. They use the selection of the symmetric control at $C \leq 20$

(Section 6.3) and are re-tuned at each larger budget (Section 5.2). The selection changed in 27 of 48 arm and budget pairs, 18 for TMR and 9 for QM, and every changed selection uses $s = 1.5$. The sweep has two vanilla baselines. The original TM keeps its $C = 20$ setting, as the modified arms do. The re-tuned TM uses TM_{ext} (Section 6.3) at $C \leq 20$ and is re-tuned at each larger budget (Section 5.2). The sweep is descriptive: it has no hypothesis test. Fig. 3 plots the mean test accuracy against C. Table 8 gives, for the arm chosen by the density rule at $C = 5$ and 10, the smallest vanilla budget whose mean accuracy is at least as high.

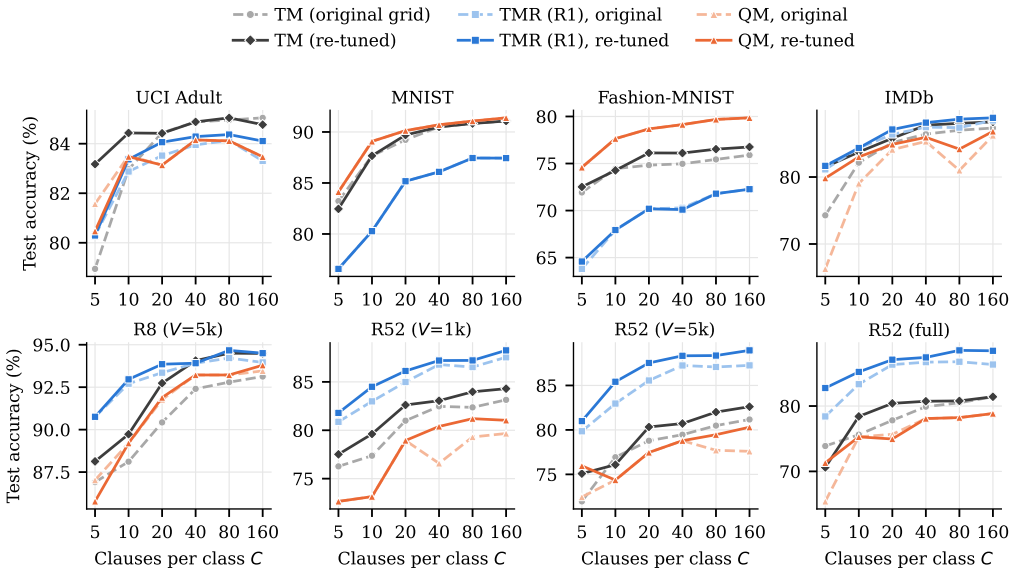


Figure 3: Clause-budget sweep: mean test accuracy over seeds 101 to 103 against clauses per class C (log scale). TMR, QM and the original TM: Study 1 settings at $C \leq 20$, the $C = 20$ setting with $T = C/2$ at $C \geq 40$. Re-tuned TM: TM_{ext} at $C \leq 20$, re-tuned per budget at $C \geq 40$. Dashed TMR and QM: original settings. Solid TMR and QM: re-tuned, with the selection of the symmetric control at $C \leq 20$ and re-tuned per budget at $C \geq 40$. Where the re-tuned selection equals the original one, the solid curve covers the dashed one.

Against the original TM, the rule arm with 5 clauses per class matches a vanilla TM with 2 to 16 times as many clauses. Against the re-tuned TM, the ratio falls to 1 to 4. Because the budget grid doubles, the true ratio lies between half the tabulated value and the value itself. On IMDb and UCI Adult the re-tuned TM with 5 clauses already matches the rule arm, so the rule arm saves no clauses there. With 10 clauses per class, the rule arm stays ahead of the re-tuned TM with 160 clauses on three encodings. R1 leads by 1.9 pp on R52 with all words and by 0.4 pp on

Table 8: Smallest vanilla clause budget per class (TM C needed) whose mean test accuracy reaches that of the rule arm with 5 or 10 clauses per class. Orig.: original TM. Re-t.: re-tuned TM. Mean of seeds 101 to 103. >160: not reached in the sweep. Second line (blue): the rule arm re-tuned in the symmetric control (Section 6.3).

	Dens.	Rule	Rule arm, $C = 5$			Rule arm, $C = 10$		
				TM C needed			TM C needed	
Dataset	(%)	arm	Acc.	orig.	re-t.	Acc.	orig.	re-t.
R52 (full)	0.38	R1	78.41	40	10	83.34	>160	>160
re-tuned			82.76	>160	>160	85.22	>160	>160
R8 (V=5k)	0.67	R1	90.76	40	20	92.71	80	20
re-tuned			90.76	40	20	92.96	160	40
R52 (V=5k)	0.68	R1	79.85	80	20	82.96	>160	>160
re-tuned			80.99	160	80	85.42	>160	>160
R52 (V=1k)	1.35	R1	80.87	20	20	82.99	160	40
re-tuned			81.80	40	20	84.49	>160	>160
IMDb	2.10	R1	81.14	10	5	84.34	20	20
re-tuned			81.67	10	10	84.34	20	20
UCI Adult	13.49	QM	81.57	10	5	83.51	20	10
re-tuned			80.47	10	5	83.48	20	10
MNIST	15.14	QM	84.13	10	10	89.08	20	20
re-tuned			84.13	10	10	89.08	20	20
Fashion-MNIST	54.07	QM	74.59	20	20	77.64	>160	>160
re-tuned			74.59	20	20	77.64	>160	>160

R52 with 5,000 words. QM leads by 0.9 pp on Fashion-MNIST. On the other five encodings the re-tuned TM catches up at 10 to 40 clauses. With the rule arm also re-tuned (blue lines in Table 8), the rule arm with 5 clauses per class matches a re-tuned vanilla TM with 1 to 16 times as many clauses on seven encodings. On the eighth, R52 with all words, R1 with 5 clauses per class stays ahead of the re-tuned TM with 160 clauses, by 1.3 pp. With 10 clauses per class, the re-tuned rule arm stays ahead of the re-tuned TM with 160 clauses on four encodings: R52 with all words (3.8 pp), R52 with 5,000 words (2.8 pp), Fashion-MNIST (0.9 pp) and R52 with 1,000 words (0.2 pp).

Across budgets, the re-tuned TM changes the picture on several encodings. On the three R52 encodings, R1 stays ahead at every budget, by 2.4 to 7.8 pp. On Fashion-MNIST, QM stays ahead at every budget, by 2.1 to 3.4 pp. On R8, R1 leads by 0.6 to 3.0 pp up to 20 clauses and falls behind by 0.2 to 0.5 pp from 40 clauses on. On MNIST, QM leads by 1.4 to 1.7 pp at 5 and 10 clauses and by only 0.2 to 0.4 pp from 20 clauses on. On IMDb, R1 and the re-tuned TM differ by at most 0.6 pp at every budget. On UCI Adult, the re-tuned TM is ahead of QM at every budget, by 0.7 to 1.6 pp. The family that the rule does not choose loses at most budgets: R1 on MNIST and Fashion-MNIST, and QM on R52. QM is also less stable across budgets on sparse text, for example 85.3, 81.0 and 86.2 % on IMDb at 40, 80 and 160 clauses.

With TMR and QM also re-tuned, the comparison with the re-tuned TM changes as follows. On the three R52 encodings, R1 stays ahead at every budget, by 3.2 to 12.1 pp. On R8, R1 leads by 1.1 to 3.2 pp up to 20 clauses and differs from the re-tuned TM by at most 0.2 pp from 40 clauses on. On IMDb, R1 is ahead of the re-tuned TM at every budget, by 0.2 to 1.3 pp. On UCI Adult, the re-tuned TM stays ahead of QM at every budget, by 0.7 to 2.7 pp. On MNIST and Fashion-MNIST, re-tuning does not change the QM selection at any budget, so the QM results above hold. The family that the rule does not choose still loses at most budgets. R1 trails by 3.4 to 7.9 pp on MNIST and Fashion-MNIST. QM on R52 leads by 0.9 and 0.7 pp at 5 clauses with 5,000 and all words, and trails by 1.7 to 6.5 pp at every other budget. On IMDb, QM trails the re-tuned TM at every budget, by 0.7 to 3.9 pp, and still drops at 80 clauses: 85.9, 84.2 and 86.8 % at 40, 80 and 160 clauses.

In AP(TM), area and the energy of each operation scale with the number of rows [3]. An F-fold clause reduction at equal accuracy therefore gives an F-fold smaller clause array and $1/F$ of the search energy per inference. Training latency per class update does not depend on the row count, so, relative to AP(TM) without a literal budget, it rises by 36 to 44 % whatever F is. Against a vanilla TM that

already uses a literal budget (Section 4.5), R1 adds no training cycles and QM at most 80 per class update, so the training energy falls by nearly F. Relative to AP(TM) without a budget, and if each added cycle costs as much energy as a base cycle, the training energy per class update falls by $F/1.437$ to $F/1.358$: 1.4 to 1.5 times for $F = 2$ and 2.8 to 2.9 times for $F = 4$. Against the re-tuned TM, F is 1 to 4 at 5 clauses per class (Table 8), and 1 to 16 on seven encodings when the rule arm is also re-tuned. The savings are therefore smaller than against the original TM, and they vanish where $F = 1$. These are projections from the analytical model and the accuracy means above, not measurements.

7. Related work

TM feedback variants. The literal budget of [7] redirects Type I feedback to Type Ib when a clause grows too large, but Type Ib remains independent of the literal value. Granmo et al. [35] prune literals with a Markov-boundary-guided automaton. Focused negative sampling [36] changes how the non-target class is drawn. The sparse and contracting TMs [37, 38] change how excluded literals are stored and keep value-independent forgetting. Drop clause [39] and integer-weighted clauses [40] act on clauses, not on Type Ib. Morris et al. [41] replace the class sum with a clause-local count for parallel hardware. TRIPOD [42] replaces the two automata of a literal pair with one three-action automaton. The closest rule to R1 is the probabilistic concept learning scheme of Belaid et al. [43]. On positive samples it always pushes false literals toward exclusion and pushes true literals toward inclusion only with a probability below one. Its feedback does not depend on the clause output, it never leaves a literal unchanged, and it serves a convergence analysis rather than a training rule. We found no TM variant that restricts Type Ib forgetting of a non-firing clause to its false literals.

Relaxed clauses. Rule learners with m-of-n conditions [44, 45] and relaxed CNF classifiers [46] fire when an absolute number of conditions holds. The Fuzzy-Pattern TM [32], an arXiv preprint, evaluates clauses fuzzily. A clause with $|I_j|$ included literals scores $\min(|I_j|, L_F)$ minus its number of failed literals, where L_F is a fixed hyperparameter, and contributes this score to the vote while it is positive. It reports higher accuracy with very few clauses than the binary-output clauses studied here, and the runs specified in our Study 2 protocol agree. A graded vote needs a multi-level mismatch count per row. A match-line sense amplifier with one reference does not provide it, so an AP would need several threshold searches or a per-row counter for each inference. We therefore do not report it as a baseline.

Our QM clause keeps a binary output, uses a tolerance proportional to the clause length, and maps to a single threshold search.

TM hardware. TM training and inference have been implemented on FPGAs [47, 48, 49], and TM inference in a 65 nm ASIC [50] and in a Y-Flash in-memory architecture [2]. AP(TM) [3] maps both inference and training onto an associative processor [4]. Approximate-search CAMs with Hamming-distance tolerance have been built for DNA classification [5, 10] and demonstrated in silicon with a tunable threshold [6]. To our knowledge, they have not been used for TM clause evaluation.

8. Discussion and limitations

What the evidence supports. With $s = 1.5$ offered to all arms, R1 gains ~~3.4~~1.5 to 17.2 pp in 19 sparse cells and loses only on dense inputs. QM gains 1.2 to 3.5 pp on MNIST and Fashion-MNIST and 6.8 to 10.9 pp on Sector. Against the stronger baseline, with the modified arms not re-tuned, two effects remain. R1 gains 3.0 to 11.7 pp on sparse text with many classes: R52, R8, RCV1 and Sector. QM, alone or with R1, gains 1.2 to 5.5 pp on dense image and signal inputs: MNIST, Fashion-MNIST and UCI HAR. It also gains 1.0 to 1.7 pp on USPS in Study 2. ~~With $s = 1.5$ offered to all arms, R1 gains 3.4 to 17.2 pp in 19 sparse cells and loses only on dense inputs. QM gains 1.2 to 3.5 pp on MNIST and Fashion-MNIST and 6.8 to 10.9 pp on Sector.~~ In the descriptive budget sweep, both effects persist to 160 clauses per class against the re-tuned TM on R52 and Fashion-MNIST, but not on R8 (Section 6.5). With TMR and QM also re-tuned at each budget, the R1 lead on R52 grows to 3.2 to 12.1 pp, and on R8 R1 stays within 0.2 pp of the re-tuned TM from 40 clauses on. Neither effect follows input density reliably. The density rule fails on new datasets, R1 loses on short-text sets, and QM loses on Letter. Density alone therefore does not select the modification. Both should be validated against a well-tuned vanilla TM on the target data. R1 lost heavily on the one-hot UCI Phishing set, our only one-hot encoding, and QM should be used on sparse text only after validation.

Baseline tuning. The largest single effect in this study is the tuning grid of the vanilla TM, in s and L together. We did not separate the two. Adding $s = 1.5$ and $L = 32$ changed the vanilla selection in 57 of 60 cells and removed most of the gains measured against the original baseline, by as much as 28 pp. All arms shared the original s grid. ~~The modified arms might gain from the finer grid as well, and we did not test this (Section 6.3).~~ Offering the finer s grid to R1 as well raised its significant wins against TM_{ext} in Study 1 from 10 to 19, and its median gain from

+0.15 to +1.61 pp (Section 6.3). Most of the loss of R1 gains under the baseline controls was therefore caused by the unequal grids. We report the original and the controlled contrasts side by side so that readers can judge both.

Scope. The hypothesis tests concern small budgets of 5 to 20 clauses per class, the regime this paper targets. The sweep to 160 clauses per class covers eight encodings with three seeds and is descriptive. In earlier exploratory runs outside these protocols, at the clause budgets of AP(TM) with reduced epochs, QM gave no significant gain on MNIST, Fashion-MNIST and UCI Adult. On IMDb it was 1.8 pp below the vanilla TM in a single 8-epoch run. On Fashion-MNIST its lead over both vanilla baselines persists to 160 clauses per class (Fig. 3). We did not measure where it vanishes between that budget and those of AP(TM). Absolute accuracy on Sector and 20 Newsgroups is low at small budgets. All runs use 40 epochs, and training has not converged on sparse text: between epochs 30 and 40 the validation accuracy of the test runs on sparse cells still rises by a median of 0.15 to 0.9 pp, depending on the arm. Rankings could shift with longer training. On the short texts of Emotion and TREC-6, the standard deviation of R1 across seeds is 7.0 to 12.0 pp. We have no explanation for the loss of R1 on one-hot UCI Phishing. Density is confounded with data type: all sparse sets are bag-of-words text, and all dense sets are thresholded image, signal or tabular features. No dataset between 0.3 and 42 % density tested the rule. The baseline controls were designed after the first two studies, in response to a review. ~~They strengthen only the vanilla TM, and its best specificity lies at the edge of the extended grid in most cells.~~ In the symmetric control, the best specificity of every arm lies at the edge of the extended grid in most cells.

Hardware. The costs of Section 4 are projections from the AP(TM) analytical model. The energy and latency of a threshold compare relative to an exact compare are assumed equal and must be confirmed at circuit level. The hardware clamps the QM tolerance at $\tau_{\max} = 5$, while the accuracy results use the unclamped rule. The two agree for clauses of up to 59 or 119 literals. This covers all inspected checkpoints in the Study 1 cells where QM wins but one, where the largest clause exceeded the limit by one literal. Only two seeds were inspected, and we did not re-run the experiments with the clamp. The training overhead of 36 to 44 % comes from the literal-budget counter. An incremental counter update could reduce it but was not evaluated. The ablation of Section 6.4 shows that R1 and QM cannot avoid this overhead by dropping the budget.

9. Conclusion

Relative qualified-majority clauses and targeted Type Ib forgetting map onto an associative processor with an approximate-search CAM with no added inference cycles and at most 0.94 % wider rows. Their training overhead comes from the literal budget. With R1, QM and the vanilla TM all offered a smaller specificity, and the vanilla TM also a smaller literal budget, R1 wins 19 of 48 cells, all on sparse text, and loses 8, all dense, and a density rule fixed in advance (R1 on sparse inputs, QM on dense ones) is not confirmed on six unseen datasets. Against the vanilla TM as first tuned, R1 wins 24 of 48 cells and a density rule fixed in advance wins 5 of 12 cells on six unseen datasets with no loss. Against a vanilla TM tuned over a smaller specificity value and literal budget, with the modified arms not re-tuned, most of the R1 gains and the density rule do not hold. R1 keeps gains of 3.0 to 11.7 pp on sparse many-class text, and QM, alone or with R1, keeps gains of 1.0 to 5.5 pp on dense image and signal data, but the density rule is not confirmed out of sample. ~~When R1 and QM are offered the same smaller specificity, R1 wins 19 of 48 cells against this baseline, all on sparse text, and loses 8, all dense. The density rule is still not confirmed out of sample.~~ Neither modification works without the literal budget, so its counter is a necessary part of their hardware cost. At 5 clauses per class, the rule arm matches a re-tuned vanilla TM with 1 to 4 times as many clauses, against 2 to 16 times for the original one, again without re-tuning the rule arm. With the rule arm also re-tuned, the ratio is 1 to 16 on seven encodings, and on R52 with all words the re-tuned vanilla TM with 160 clauses per class does not reach R1 with 5. For in-memory TM design, these results favor careful tuning of the vanilla baseline first, and per-dataset validation of either modification as a low-cost hardware option.

Data and code availability

Code and results are available at an anonymized repository: `https://anonymous.4open.science/r/QMTM-1991/`.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) to draft and edit the manuscript text, to write the experiment, analysis and plotting code, to run the experiments and to check the references. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Appendix A. Per-cell results

References

[1] O.-C. Granmo, The Tsetlin machine – a game theoretic bandit driven approach to optimal pattern recognition with propositional logic (2018). arXiv:1804.01508.

[2] O. Ghazal, W. Wang, S. Kvatinsky, F. Merchant, A. Yakovlev, R. Shafik, IMPACT: In-memory ComPuting architecture based on Y-FlAsh technology for coalesced Tsetlin machine inference, Philosophical Transactions of the Royal Society A 383 (2288) (2025) 20230393. doi:10.1098/rsta.2023.0393.

[3] H. Shrot, Y. Rudin, A. Fish, L. Yavits, AP(TM): Associative processor based Tsetlin machine, Expert Systems with Applications 334 (2026) 134830. doi:10.1016/j.eswa.2026.134830.

[4] L. Yavits, A. Morad, R. Ginosar, Computer architecture with associative processor replacing last-level cache and SIMD accelerator, IEEE Transactions on Computers 64 (2) (2015) 368–381. doi:10.1109/TC.2013.220.

[5] E. Garzón, R. Golman, Z. Jahshan, R. Hanhan, N. Vinshtok-Melnik, M. Lanuzza, A. Teman, L. Yavits, Hamming distance tolerant content-addressable memory (HD-CAM) for DNA classification, IEEE Access 10 (2022) 28080–28093. doi:10.1109/ACCESS.2022.3158305.

[6] E. Garzón, E. Rechef, R. Golman, O. Harel, Y. Harary, P. Snapir, M. Lanuzza, A. Teman, L. Yavits, A 128-kbit approximate search-capable content-addressable memory (CAM) with tunable Hamming distance, IEEE Journal of Solid-State Circuits 60 (8) (2025) 3009–3019. doi:10.1109/JSSC.2025.3529715.

Table A.9: Study 1 test accuracy (%, mean of seeds 101 to 105) per cell, ordered by density. Best QM arm: the QM-family arm selected on validation. Δ_{R1} = TMR − TM. Δ_{QM} = QM − TM. Δ_{best} = best QM arm − best non-QM arm (best non-QM arm not shown). Bold: significant after Holm correction over 48 cells.

Dataset	C	TM	TMR	QM	Best QM arm	Δ_{R1}	Δ_{QM}	Δ_{best}
20NG (V=20k)	5	39.03	48.21	39.42	47.58 (QPR)	+9.18	+0.39	−0.63
	10	46.38	60.07	45.66	58.34 (QPR)	**+13.69**	−0.72	−1.73
R52 (full)	5	73.17	79.80	67.43	79.84 (QPR)	**+6.63**	−5.74	+0.05
	10	75.89	82.82	74.71	83.15 (QPR)	**+6.93**	−1.18	+0.33
	20	77.77	85.35	75.41	84.89 (QPR)	**+7.58**	−2.35	−0.46
Sector (V=1k)	5	20.17	16.36	14.37	18.02 (QPR)	−3.81	−5.80	−2.15
	10	20.52	22.16	20.78	21.40 (QPR)	+1.64	+0.26	−0.75
Sector (V=5k)	5	22.11	30.03	21.10	31.24 (QPR)	**+7.92**	−1.00	+1.21
	10	25.58	48.18	29.19	48.79 (QPR)	**+22.59**	+3.61	+0.61
Sector (V=20k)	5	22.06	33.05	19.91	30.45 (QMR)	**+10.98**	−2.15	−2.59
	10	28.48	52.04	32.63	51.94 (QPR)	**+23.56**	+4.15	−0.10
Dorothea (V=5k)	5	92.29	92.74	92.57	92.34 (QPR)	+0.46	+0.29	+0.06
	10	93.60	93.03	93.60	93.83 (QP)	−0.57	+0.00	+0.80
R8 (V=5k)	5	87.03	90.65	86.85	88.81 (QMR)	**+3.62**	−0.18	−1.84
	10	88.34	92.99	89.05	92.41 (QPR)	**+4.65**	+0.72	−0.58
	20	90.60	93.40	92.06	91.70 (QPR)	**+2.80**	+1.46	−1.70
R52 (V=5k)	5	72.50	79.79	72.58	79.75 (QPR)	**+7.29**	+0.08	−0.04
	10	75.90	82.16	75.07	81.12 (QMR)	**+6.26**	−0.83	−1.04
	20	78.12	84.61	76.22	85.08 (QPR)	**+6.49**	−1.90	+0.47
LEDGAR (V=5k)	5	61.07	59.87	61.61	61.61 (QM)	−1.20	+0.54	+0.54
	10	66.81	67.97	69.22	69.23 (QMR)	+1.16	+2.41	+1.26
20NG (V=5k)	5	41.33	48.08	39.27	49.62 (QPR)	**+6.76**	−2.06	+1.53
	10	47.93	62.02	44.09	61.78 (QPR)	**+14.09**	−3.85	−0.25
Dorothea (full)	5	91.54	91.94	92.63	92.63 (QM)	+0.40	+1.09	+0.69
	10	90.57	92.97	93.71	93.71 (QM)	+2.40	+3.14	+0.74
RCV1 (V=5k)	5	51.91	63.56	53.23	65.45 (QMR)	**+11.65**	+1.32	+1.89
	10	61.23	71.69	58.41	71.69 (QPR)	**+10.46**	−2.82	+0.00
20NG (V=1k)	5	41.88	42.83	40.65	48.51 (QPR)	+0.95	−1.23	+5.68
	10	50.70	60.18	50.42	59.80 (QPR)	**+9.48**	−0.28	−0.38
R52 (V=1k)	5	76.08	81.04	73.69	80.36 (QPR)	**+4.96**	−2.40	−0.69
	10	77.72	83.11	74.79	82.14 (QPR)	**+5.40**	−2.93	−0.97
	20	80.48	85.45	78.82	86.14 (QPR)	**+4.97**	−1.66	+0.69
IMDb	5	74.24	80.56	65.10	80.62 (QPR)	**+6.32**	−9.14	+0.06
	10	82.39	84.42	79.68	82.72 (QPR)	**+2.03**	−2.72	−1.71
	20	85.16	86.11	81.36	86.06 (QPR)	+0.94	−3.80	−0.04
UCI Adult	5	79.84	80.60	81.28	81.28 (QM)	+0.76	+1.44	+0.68
	10	83.30	83.11	83.27	83.27 (QM)	−0.19	−0.04	−0.04
	20	84.47	83.23	83.16	83.96 (QP)	−1.24	−1.31	+0.73
MNIST	5	83.14	75.27	84.19	84.19 (QM)	−7.87	+1.05	+1.05
	10	87.70	80.71	88.93	88.93 (QM)	**−6.99**	**+1.23**	**+1.23**
	20	89.30	85.43	90.09	90.09 (QM)	**−3.87**	+0.80	+0.80
UCI Phishing	5	88.95	69.03	88.33	88.63 (QP)	**−19.92**	−0.62	−0.32
	10	90.96	69.39	90.49	90.49 (QM)	**−21.57**	−0.47	−0.47
UCI HAR	5	84.77	85.27	87.95	90.60 (QMR)	+0.50	+3.18	**+5.33**
	10	87.17	88.10	90.18	91.99 (QMR)	+0.92	**+3.01**	**+3.90**
Fashion-MNIST	5	71.78	64.40	74.71	74.71 (QM)	−7.38	**+2.93**	**+2.93**
	10	74.45	68.29	77.73	77.73 (QM)	**−6.16**	**+3.28**	**+3.28**
	20	74.89	70.53	78.74	78.74 (QM)	**−4.37**	**+3.84**	**+3.84**

[7] K. D. Abeyrathna, A. A. O. Abouzeid, B. Bhattarai, C. Giri, S. Glimsdal, O.-C. Granmo, L. Jiao, R. Saha, J. Sharma, S. A. Tunheim, X. Zhang, Building concise logical patterns by constraining Tsetlin machine clause size, in: Proc. 32nd Int. Joint Conf. Artificial Intelligence (IJCAI), 2023, pp. 3395–3403. doi:10.24963/ijcai.2023/378.

[8] O.-C. Granmo, pyTsetlinMachine, https://github.com/cair/pyTsetlinMachine, python package, version 0.6.6 (2024).

[9] K. Pagiamtzis, A. Sheikholeslami, Content-addressable memory (CAM) circuits and architectures: A tutorial and survey, IEEE Journal of Solid-State Circuits 41 (3) (2006) 712–727. doi:10.1109/JSSC.2005.864128.

[10] Z. Jahshan, I. Merlin, E. Garzón, L. Yavits, DASH-CAM: Dynamic approximate SearcH content addressable memory for genome classification, in: Proc. 56th Annual IEEE/ACM Int. Symp. Microarchitecture (MICRO), ACM, 2023, pp. 1453–1465. doi:10.1145/3613424.3614262.

[11] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, Proceedings of the IEEE 86 (11) (1998) 2278–2324. doi:10.1109/5.726791.

[12] H. Xiao, K. Rasul, R. Vollgraf, Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms (2017). arXiv:1708.07747.

[13] R. Kohavi, Scaling up the accuracy of Naive-Bayes classifiers: a decision-tree hybrid, in: Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD-96), AAAI Press, 1996, pp. 202–207.

[14] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, C. Potts, Learning word vectors for sentiment analysis, in: Proc. 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, Association for Computational Linguistics, Portland, Oregon, USA, 2011, pp. 142–150.

[15] D. Anguita, A. Ghio, L. Oneto, X. Parra, J. L. Reyes-Ortiz, A public domain dataset for human activity recognition using smartphones, in: Proc. European Symp. Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN), i6doc.com, Bruges, Belgium, 2013, pp. 437–442.

[16] R. M. Mohammad, F. Thabtah, L. McCluskey, Predicting phishing websites based on self-structuring neural network, Neural Computing and Applications 25 (2) (2014) 443–458. doi:10.1007/s00521-013-1490-z.

[17] A. Cardoso-Cachopo, Improving methods for single-label text categorization, Ph.D. thesis, Instituto Superior Técnico, Universidade Técnica de Lisboa (2007).

[18] K. Lang, NewsWeeder: Learning to filter netnews, in: Proc. Int. Conf. Machine Learning (ICML), 1995, pp. 331–339. doi:10.1016/B978-1-55860-377-6.50048-7.

[19] J. D. M. Rennie, R. Rifkin, Improving multiclass text classification with the support vector machine, AI Memo AIM-2001-026, Massachusetts Institute of Technology (Oct. 2001).
URL http://hdl.handle.net/1721.1/7241

[20] D. D. Lewis, Y. Yang, T. G. Rose, F. Li, RCV1: A new benchmark collection for text categorization research, Journal of Machine Learning Research 5 (2004) 361–397.

[21] D. Tuggener, P. von Däniken, T. Peetz, M. Cieliebak, LEDGAR: A large-scale multi-label corpus for text classification of legal provisions in contracts, in: Proc. 12th Language Resources and Evaluation Conf. (LREC), European Language Resources Association, Marseille, France, 2020, pp. 1235–1241.

[22] I. Chalkidis, A. Jana, D. Hartung, M. Bommarito, I. Androutsopoulos, D. Katz, N. Aletras, LexGLUE: A benchmark dataset for legal language understanding in English, in: Proc. 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Dublin, Ireland, 2022, pp. 4310–4330. doi:10.18653/v1/2022.acl-long.297.

[23] I. Guyon, S. Gunn, A. Ben-Hur, G. Dror, Result analysis of the NIPS 2003 feature selection challenge, in: Advances in Neural Information Processing Systems 17 (NIPS 2004), 2004, pp. 545–552.

[24] S. Bird, E. Klein, E. Loper, Natural Language Processing with Python, O'Reilly Media, Inc., 2009.

[25] C.-C. Chang, C.-J. Lin, LIBSVM: A library for support vector machines, ACM Transactions on Intelligent Systems and Technology 2 (3) (2011) 27:1–27:27. doi:10.1145/1961189.1961199.

[26] X. Zhang, J. Zhao, Y. LeCun, Character-level convolutional networks for text classification, in: Advances in Neural Information Processing Systems 28 (NIPS 2015), 2015, pp. 649–657.

[27] X. Li, D. Roth, Learning question classifiers, in: Proc. 19th Int. Conf. Computational Linguistics (COLING), 2002.

[28] E. Saravia, H.-C. T. Liu, Y.-H. Huang, J. Wu, Y.-S. Chen, CARER: Contextualized affect representations for emotion recognition, in: Proc. 2018 Conf. Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, Brussels, Belgium, 2018, pp. 3687–3697. doi:10.18653/v1/D18-1404.

[29] J. J. Hull, A database for handwritten text recognition research, IEEE Transactions on Pattern Analysis and Machine Intelligence 16 (5) (1994) 550–554. doi:10.1109/34.291440.

[30] P. W. Frey, D. J. Slate, Letter recognition using Holland-style adaptive classifiers, Machine Learning 6 (2) (1991) 161–182. doi:10.1007/BF00114162.

[31] D. Michie, D. J. Spiegelhalter, C. C. Taylor (Eds.), Machine Learning, Neural and Statistical Classification, Ellis Horwood Series in Artificial Intelligence, Ellis Horwood, Chichester, 1994.

[32] A. Hnilov, Fuzzy-pattern Tsetlin machine (2025). arXiv:2508.08350.

[33] B. L. Welch, The generalization of 'Student's' problem when several different population variances are involved, Biometrika 34 (1-2) (1947) 28–35. doi:10.1093/biomet/34.1-2.28.

[34] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.
URL http://www.jstor.org/stable/4615733

[35] O.-C. Granmo, P.-A. Andersen, L. Jiao, X. Zhang, C. Blakely, G. T. Berge, T. Tveit, Learning minimalistic Tsetlin machine clauses with Markov boundary-guided pruning (2023). arXiv:2309.06315.

[36] S. Glimsdal, R. Saha, B. Bhattarai, C. Giri, J. Sharma, S. A. Tunheim, R. K. Yadav, Focused negative sampling for increased discriminative power in Tsetlin machines, in: Proc. Int. Symp. Tsetlin Machine (ISTM), 2022, pp. 73–80. doi:10.1109/ISTM54910.2022.00021.

[37] S. Østby, T. M. Brambo, S. Glimsdal, The sparse Tsetlin machine: Sparse representation with active literals (2024). arXiv:2405.02375.

[38] B. Bhattarai, O.-C. Granmo, L. Jiao, P.-A. Andersen, S. A. Tunheim, R. Shafik, A. Yakovlev, Contracting Tsetlin machine with absorbing automata (2023). arXiv:2310.11481.

[39] J. Sharma, R. Yadav, O.-C. Granmo, L. Jiao, Drop clause: Enhancing performance, robustness and pattern recognition capabilities of the Tsetlin machine, in: Proc. AAAI Conf. Artificial Intelligence, 2023, pp. 13547–13555. doi:10.1609/aaai.v37i11.26588.

[40] K. D. Abeyrathna, O.-C. Granmo, M. Goodwin, Extending the Tsetlin machine with integer-weighted clauses for increased interpretability, IEEE Access 9 (2021) 8233–8248. doi:10.1109/ACCESS.2021.3049569.

[41] J. Morris, A. Rafiev, F. Xia, R. Shafik, A. Yakovlev, A. Brown, An alternate feedback mechanism for Tsetlin machines on parallel architectures, in: Proc. Int. Symp. Tsetlin Machine (ISTM), 2022, pp. 53–56. doi:10.1109/ISTM54910.2022.00018.

[42] Y. Rudin, M. Yemini, O. Keren, A. Fish, TRIPOD: three-action learning automaton for Tsetlin machine learning, International Journal of Systems SciencePublished online 10 June 2026 (2026). doi:10.1080/00207721.2026.2674268.

[43] M.-B. Belaid, J. Sharma, L. Jiao, O.-C. Granmo, P.-A. Andersen, A. Yazidi, Generalized convergence analysis of Tsetlin automaton based algorithms: A probabilistic approach to concept learning, in: Proc. AAAI Conf. Artificial Intelligence, 2025, pp. 15524–15531. doi:10.1609/aaai.v39i15.33704.

[44] P. M. Murphy, M. J. Pazzani, ID2-of-3: Constructive induction of M-of-N concepts for discriminators in decision trees, in: Machine Learning Proceedings 1991 (Proc. 8th Int. Workshop on Machine Learning), Morgan Kaufmann, 1991, pp. 183–187. doi:10.1016/B978-1-55860-200-7.50040-4.

[45] G. G. Towell, J. W. Shavlik, Extracting refined rules from knowledge-based neural networks, Machine Learning 13 (1) (1993) 71–101. doi:10.1007/BF00993103.

[46] B. Ghosh, D. Malioutov, K. S. Meel, Classification rules in relaxed logical form, in: Proc. 24th European Conf. Artificial Intelligence (ECAI), Frontiers in Artificial Intelligence and Applications, IOS Press, 2020. doi:10.3233/FAIA200382.

[47] S. A. Tunheim, L. Jiao, R. Shafik, A. Yakovlev, O.-C. Granmo, Convolutional Tsetlin machine-based training and inference accelerator for 2-D pattern classification, Microprocessors and Microsystems 103 (2023) 104949. doi:10.1016/j.micpro.2023.104949.

[48] S. A. Tunheim, L. Jiao, R. Shafik, A. Yakovlev, O.-C. Granmo, Tsetlin machine-based image classification FPGA accelerator with on-device training, IEEE Transactions on Circuits and Systems I: Regular Papers 72 (2) (2025) 830–843. doi:10.1109/TCSI.2024.3519191.

[49] A. Wheeldon, R. Shafik, T. Rahman, J. Lei, A. Yakovlev, O.-C. Granmo, Learning automata based energy-efficient AI hardware design for IoT applications, Philosophical Transactions of the Royal Society A 378 (2182) (2020) 20190593. doi:10.1098/rsta.2019.0593.

[50] S. A. Tunheim, Y. Zheng, L. Jiao, R. Shafik, A. Yakovlev, O.-C. Granmo, An all-digital 8.6-nJ/frame 65-nm Tsetlin machine image classification accelerator, IEEE Transactions on Circuits and Systems I: Regular Papers 73 (2) (2026) 1107–1120. doi:10.1109/TCSI.2025.3586698.